Quantifying the Utility of Disaggregated Memory in AI Systems

An Operational- and Communication-Intensity View of Compute, Memory, and I/O Ceilings

Brian D. Taylor

December 10, 2025

Abstract

Recent claims around "breakthrough" disaggregated HBM have raised expectations that remote memory fabrics could shift large-scale AI training from being memory-bound to compute-bound. This note grounds those claims in a simple, intensity-based performance model that unifies compute, memory, I/O, and network ceilings. Our scope is deliberately narrow: we ask whether disaggregated memory bandwidth alone can push a transformer training step across the compute-bound ridge point, not whether disaggregated memory has value more broadly (e.g., via added capacity, model fit, or pooling), questions we return to briefly but do not attempt to quantify. Using the roofline framework, we express kernels and training steps in terms of operational intensity (OI) and show that modern transformer workloads typically operate in the 30–80 FLOP/Byte regime. For PFLOP-class accelerators, this implies required per-device bandwidths on the order of 10–40 TB/s to reach the compute roofline, with even higher requirements for future 5–10 PFLOP devices. Existing HBM and plausible on-die SRAM tiers can raise effective bandwidth and utilization but cannot close this gap. We then introduce an I/O ceiling for disaggregated memory and show that even the most aggressive published fabrics ($\lesssim 14$ TB/s) fall one to two orders of magnitude short of the bandwidth needed if remote memory were the limiting tier, and therefore are insufficient to make transformer workloads compute-bound. Extending the same logic to distributed training, we define a communication roofline based on communication intensity (CI) and network bandwidth, and show how collective communication can become the dominant ceiling even when local memory bandwidth is ample. Overall, the framework provides a compact set of equations for diagnosing which ceiling is active and for evaluating architectural proposals, including additional HBM stacks, SRAM tiers, disaggregated memory, or network upgrades, in terms of the operational and communication intensities of real workloads.

Contents

1 Introduction

Recently there has been a surge of excitement around companies claiming breakthroughs in disaggregated HBM, including at least one high-profile acquisition based on the idea that remote HBM could fundamentally shift AI systems from being memory-bound to compute-bound. Given these claims, it is important to ground expectations in first-principles scaling laws that describe how modern AI workloads actually consume compute and memory.

For today's large transformer models, the amount of arithmetic performed per byte of memory accessed, known as operational intensity (OI), is relatively low, typically in the 30–80 FLOP/Byte range. To fully utilize PFLOP-class accelerators, a memory system must deliver bandwidth equal to compute/OI, which works out to 10–40 TB/s per device. Current HBM falls short of this threshold, and even adding large, fast SRAM tiers cannot reach the required effective bandwidth. Disaggregated memory fabric links are even further behind, constrained to only a fraction of the bandwidth required to feed modern accelerators at peak. As shown in Table 1 and Figure 1, peak compute has grown roughly 36× since Volta, while HBM bandwidth has grown only about 9× over the same four generations, so the compute-to-bandwidth gap has itself widened by roughly 4×, and continues to widen with each new generation.

GPU Generation	**Peak Compute**	**HBM Bandwidth**	**Relative Growth**
Volta (V100)	0.125 PFLOP/s	0.9 TB/s	Baseline (1.0× vs. 1.0×)
Ampere (A100)	0.312 PFLOP/s	1.6 TB/s	2.5× compute / 1.8× memory
Hopper (H100)	1.0 PFLOP/s	3.35 TB/s	8.0× compute / 3.7× memory
Blackwell (GB200)	4.5 PFLOP/s	8.0 TB/s	36× compute / 8.9× memory

Table 1: Representative peak compute throughput and HBM bandwidth across four GPU generations, normalized to Volta (V100) as baseline. All figures are dense (non-sparse) FP16/BF16 throughput for consistency across generations; the Blackwell row reflects the dual-die GB200 superchip, matching the device modeled in the worked example of Section 6.1.

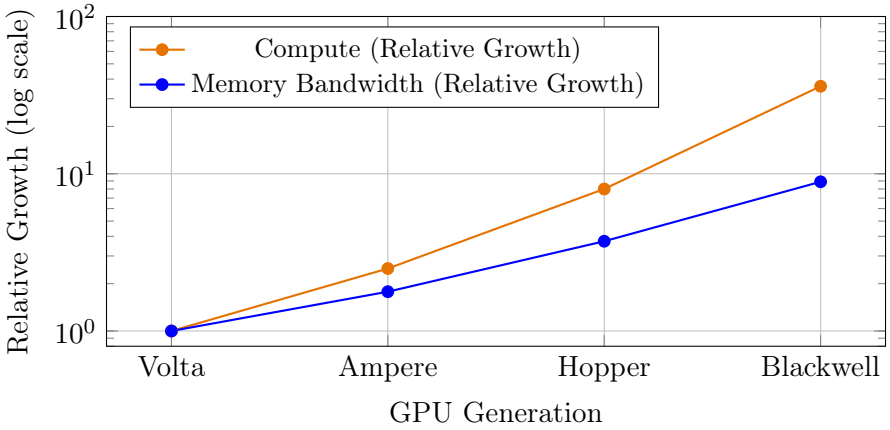

Figure 1: Relative growth of compute and HBM bandwidth as a function of GPU generation, plotted directly from the values in Table 1 (Volta = 1.0× baseline).

We will show that increasing on-package bandwidth (HBM) and adding fast on-die or stacked SRAM tiers can improve utilization, but neither is sufficient to make modern transformer training compute-bound under realistic step-level operational intensities. Disaggregated memory, as we

model it here, primarily increases addressable capacity; with current and near-term I/O ceilings it is not expected to move typical transformer workloads into a compute-bound regime. Throughout this note, we therefore focus narrowly on the minimum bandwidth required for modern transformer workloads to become compute-bound at the device level. To maintain analytical clarity, we intentionally omit several practical considerations that further reduce achievable performance in deployed systems, such as latency and RTT sensitivity, bandwidth-delay product constraints, queuing and prefetch depth, granularity and coherence effects, and memory reliability and error rates. Each of these factors lowers effective delivered bandwidth relative to the idealized values considered here; accordingly, the bounds we derive should be interpreted as roofline upper limits rather than predictions of realized utilization. Likewise, we do not attempt to quantify capacity-driven system-level benefits of disaggregated memory (e.g., model fit, memory pooling, or reduced stranding), which are orthogonal to the compute-bound question considered here. Finally, our numerical examples (Table 1 and Sections 6 and 7) are calibrated to NVIDIA-style GPUs with off-chip HBM stacked behind a smaller on-die SRAM tier. Architectures that respond to the same memory wall differently, including wafer-scale designs that keep an entire model in very large on-die SRAM with no off-chip HBM in the critical path, or TPU-style pods built around a different optical-circuit-switched fabric, are not modeled here and may face a different balance of ceilings than the one derived below.

2 Purpose and Overview

Here we provide a compact, reusable set of equations for reasoning about where large-scale AI training is fundamentally limited:

- By device compute throughput (FLOP/s),
- By local memory bandwidth (e.g., HBM and SRAM),
- By off-package I/O (e.g., NVLink/PCIe/CXL),
- Or by distributed communication (e.g., all-reduce over a training network).

We use the **roofline model** [1] as the organizing framework. In that model, a kernel or training step is described by:

- Its *operational intensity* (OI): FLOPs executed per byte moved from the limiting memory tier.
- The machine's *machine balance* (MB): peak FLOPs per byte per second of sustained memory bandwidth for that tier.

Comparing OI to MB tells us whether a workload is *compute-bound* or *memory-bound* at a given tier. We then extend the same logic to two-tier memories (SRAM + HBM), disaggregated memory behind an I/O ceiling, and network collectives.

3 Notation (Symbols and Units)

Symbol	Meaning	Typical units
F	Floating-point operations executed	FLOP
M	Bytes moved from the limiting memory tier	Byte
OI	Operational intensity $= F/M$	FLOP/Byte
C	Peak compute throughput (device ceiling)	FLOP/s
B	Sustained bandwidth for the limiting tier	Byte/s
MB	Machine balance $= C/B$	FLOP/Byte
T	Achievable throughput (roofline-bound)	FLOP/s
U	Compute utilization $= T/C$	Dimensionless
s	Bytes per element (e.g., FP16/BF16 $\Rightarrow s = 2$)	Byte/element
B_{SRAM}	Bandwidth of SRAM tier	Byte/s
B_{HBM}	Bandwidth of HBM tier	Byte/s
h	Hit rate in fast tier (SRAM)	Dimensionless
B_{IO}	Off-package I/O bandwidth ceiling	Byte/s
B_{net}	Network bandwidth for collectives	Byte/s
N	Number of devices/ranks	Dimensionless
S	Model state communicated (e.g., gradients)	Byte
α	Collective latency term (startup / RTT dominated)	s
β	Inverse bandwidth term ($\beta \approx 1/B_{\text{net}}$)	s/Byte
T_{comp}	Compute time per step	s
T_{comm}	Communication time per step	s
CI	Communication intensity (Bytes/FLOP)	Byte/FLOP

Table 2: Notation used throughout the paper, including compute, memory, I/O, and communication parameters.

4 The Roofline Model

4.1 What is the roofline model?

The **roofline model** [1] is a simple performance model that upper-bounds the attainable throughput of a kernel or workload based on:

- The peak compute rate C of the device (FLOP/s), and
- The sustained memory bandwidth B of the limiting tier (Byte/s).

On a standard roofline plot:

- The horizontal axis is the *operational intensity* OI (FLOP/Byte).
- The vertical axis is the attained performance T (FLOP/s).

There are two ceilings:

1. A flat *compute roof* at $T = C$ (limited by arithmetic throughput).
2. A sloped *memory roof* at $T = \text{OI} \cdot B$ (limited by bandwidth).

The achievable performance is the minimum of these two:

$$T(\text{OI}) = \min\{C,\ \text{OI} \cdot B\}. \tag{1}$$

Figure 2 illustrates the roofline and ridge line concepts and outlines memory-bound and compute-bound regions.

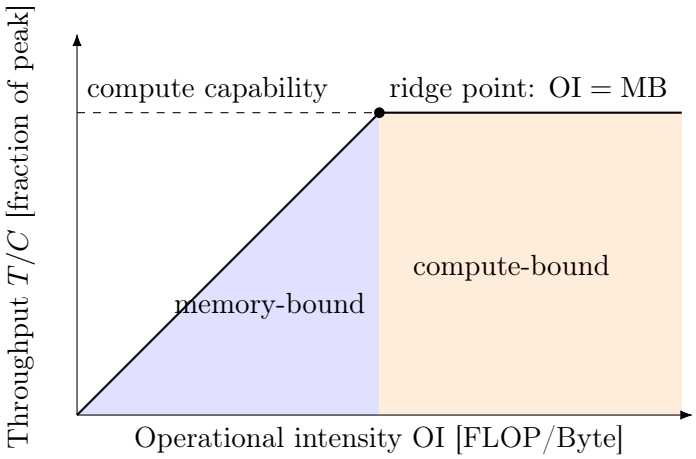

Figure 2: Roofline model: memory-bound region (blue) and compute-bound region (orange) relative to the ridge point $\mathrm{OI} = \mathrm{MB} = C/B$.

4.2 Operational intensity (OI)

Following Williams et al. [1], the **operational intensity** of a kernel or training step with respect to a given memory tier is:

$$\mathrm{OI} \triangleq \frac{F}{M} \qquad [\mathrm{FLOP/Byte}], \tag{2}$$

where F is the number of FLOPs executed and M is the number of bytes moved from the tier of interest (e.g., HBM).

Intuitively, OI measures how much arithmetic work you get per byte loaded from that tier. Higher OI means better reuse and less bandwidth pressure for a fixed amount of compute.

4.3 Machine balance (MB)

The **machine balance** for a given memory tier is:

$$\mathrm{MB} \triangleq \frac{C}{B} \qquad [\mathrm{FLOP/Byte}], \tag{3}$$

where C is peak compute throughput and B is sustained bandwidth for that tier.

In roofline terminology [1, 3], MB is the "ridge point": it is the operational intensity at which the bandwidth roof and compute roof intersect.

- If $\mathrm{OI} < \mathrm{MB}$, the workload is *memory-bound*: its throughput is limited by B, not by C.
- If $\mathrm{OI} \geq \mathrm{MB}$, the workload can be compute-bound, assuming no other ceiling (e.g., I/O or network) intervenes.

4.4 Utilization under the roofline

Compute utilization is the fraction of peak FLOPs actually delivered:

$$U \triangleq \frac{T}{C} = \min\left\{1, \frac{\mathrm{OI} \cdot B}{C}\right\}. \tag{4}$$

Equivalently, if OI < MB, then $\mathrm{OI} \cdot B/C = \mathrm{OI}/\mathrm{MB} < 1$ and utilization is proportional to OI.

4.5 Bandwidth required to become compute-bound

Rearranging the condition for compute-bound behavior,

$$\mathrm{OI} \cdot B \geq C$$

gives the required bandwidth to saturate compute at a given OI:

$$B_{\mathrm{req}} = \frac{C}{\mathrm{OI}}. \tag{5}$$

This expression is useful for quickly answering questions of the form: *"If my training block has* $\mathrm{OI} \approx 50$ *FLOP/Byte and my device is 1 PFLOP/s, how much memory bandwidth do I actually need for compute-bound training?"*

5 Example Kernels: GEMM and Transformer Blocks

5.1 GEMM (general matrix multiplications)

For a dense GEMM of shape $(M \times K) \cdot (K \times N)$, the FLOPs are:

$$F_{\mathrm{GEMM}} = 2MKN. \tag{6}$$

A common, simple traffic model (reading each input once and writing the output once) is:

$$M_{\mathrm{bytes}} \approx (MK + KN + MN) \cdot s, \tag{7}$$

where s is bytes per element (e.g., $s = 2$ for FP16/BF16). Then:

$$\mathrm{OI}_{\mathrm{GEMM}} \approx \frac{2MKN}{(MK + KN + MN) \cdot s}. \tag{8}$$

For large matrices with reasonable blocking, GEMM can reach high OI and often sits in a compute-bound regime on modern accelerators.

5.2 Transformer block (back-of-the-envelope)

A common rough approximation of FLOPs per token per layer for a transformer block (in terms of hidden size d and sequence length n) is:

$$F_{\mathrm{tok}} \approx 12d^2 + 4dn, \tag{9}$$

which follows the standard FLOP accounting for the feed-forward network (FFN) and attention (e.g., De Vries [4], the JAX transformer notes [5], and Timbers [6]).

A crude traffic proxy (ignoring caches and fusions) is:

$$M_{\mathrm{tok}} \approx 6d + 2n \qquad \text{[Bytes, up to a constant factor]}, \tag{10}$$

consistent with the per-token KV/cache and activation byte counts in Kipply [7] and related analyses. Then:

$$\mathrm{OI}_{\mathrm{tok}} \approx \frac{F_{\mathrm{tok}}}{M_{\mathrm{tok}}}. \tag{11}$$

In practice, real Transformer training and inference steps mix high-OI matmuls with lower-OI components such as softmax, layer normalization, and elementwise operations. Empirical profiling therefore finds that the effective step-level arithmetic intensity for modern LLMs is typically in the "tens of FLOPs per byte" regime, e.g., on the order of 30–80 FLOP/Byte [8, 10, 11, 9].

6 Two-Tier Memory: SRAM + HBM

Modern accelerators increasingly add a fast on-die or stacked SRAM tier in front of HBM. Let:

- B_{SRAM} be the bandwidth of the SRAM tier,
- B_{HBM} be the bandwidth of the HBM tier,
- h be the fraction of traffic (bytes) served by SRAM (the SRAM "hit rate").

A first-order, effective bandwidth model is:

$$B_{\text{eff}} \approx h \cdot B_{\text{SRAM}} + (1 - h) \cdot B_{\text{HBM}}. \tag{12}$$

Higher h moves the workload closer to a compute-bound regime by increasing B_{eff}, and thus raising the effective machine balance:

$$\text{MB}_{\text{eff}} \approx \frac{C}{B_{\text{eff}}}. \tag{13}$$

6.1 Worked example with an SRAM tier (GB200-class device)

Consider a Blackwell-class (GB200) accelerator with:

- Peak compute: $C \approx 4.5 \times 10^{15}$ FLOP/s (4.5 PFLOP/s FP16/BF16).
- HBM bandwidth: $B_{\text{HBM}} = 8$ TB/s.
- Hypothetical SRAM bandwidth: $B_{\text{SRAM}} = 32$ TB/s.

We examine a transformer block whose effective operational intensity is

$$\text{OI} \approx 50 \text{ FLOP/Byte}.$$

Step 1: Single-tier HBM (no SRAM). The machine balance for the HBM tier is:

$$\text{MB}_{\text{HBM}} = \frac{C}{B_{\text{HBM}}} = \frac{4.5 \times 10^{15}}{8 \times 10^{12}} \approx 560 \text{ FLOP/Byte}.$$

Since $\text{OI} = 50 \ll 560$, the workload is strongly memory-bound. The maximum attainable throughput from HBM is:

$$T_{\text{HBM}} = \text{OI} \cdot B_{\text{HBM}} = 50 \times 8 \times 10^{12} \text{ FLOP/s} = 4 \times 10^{14} \text{ FLOP/s},$$

which is only about 9% of the 4.5 PFLOP/s compute peak.

Step 2: Add an SRAM tier with hit rate $h = 0.75$. Assume that with an SRAM tier we can serve 75% of the bytes from SRAM:

$$h = 0.75.$$

Then the effective bandwidth is:

$$\begin{aligned} B_{\text{eff}} &\approx hB_{\text{SRAM}} + (1 - h)B_{\text{HBM}} \\ &= 0.75 \times 32 \text{ TB/s} + 0.25 \times 8 \text{ TB/s} \\ &= 24 \text{ TB/s} + 2 \text{ TB/s} \\ &= 26 \text{ TB/s}. \end{aligned}$$

The new machine balance is:

$$\mathrm{MB}_{\mathrm{eff}} = \frac{C}{B_{\mathrm{eff}}} = \frac{4.5 \times 10^{15}}{26 \times 10^{12}} \approx 1.7 \times 10^{2} \text{ FLOP/Byte}.$$

We still have $\mathrm{OI} = 50 < 170$, so the workload is still memory-bound, but less severely. The attainable throughput becomes:

$$T_{\mathrm{eff}} = \mathrm{OI} \cdot B_{\mathrm{eff}} = 50 \times 26 \times 10^{12} \text{ FLOP/s} = 1.3 \times 10^{15} \text{ FLOP/s},$$

which is about 29% of peak instead of 9%.

Step 3: Can we ever make this workload compute-bound with these tiers? To be compute-bound at $\mathrm{OI} = 50$, we would need:

$$B_{\mathrm{req}} = \frac{C}{\mathrm{OI}} = \frac{4.5 \times 10^{15}}{50} = 9 \times 10^{13} \text{ Byte/s} = 90 \text{ TB/s}.$$

Even if $h = 1$ (all traffic served from SRAM), the maximum effective bandwidth is:

$$B_{\mathrm{eff,max}} = B_{\mathrm{SRAM}} = 32 \text{ TB/s} < B_{\mathrm{req}}.$$

Therefore, no choice of h can make this particular workload compute-bound on this device, given $B_{\mathrm{SRAM}} = 32$ TB/s and $B_{\mathrm{HBM}} = 8$ TB/s. The SRAM tier substantially improves utilization (from $\sim 9\%$ to $\sim 29\%$), but the step remains memory-limited.

7 Disaggregated Memory and the I/O Ceiling

Figure 3 illustrates the resulting tiered memory hierarchy: fast on-device SRAM and HBM tiers feed into an off-device, disaggregated memory pool over a bandwidth-limited I/O or fabric link.

Now consider disaggregated memory that lives off-package (e.g., across a memory fabric) and is fed into the accelerator over a finite I/O interface with bandwidth B_{IO}. Even if the remote memory pool itself is very fast, the delivered bandwidth into the device cannot exceed:

$$B_{\mathrm{delivered}} \leq \min\{B_{\mathrm{remote\ mem}},\ B_{\mathrm{IO}}\}. \tag{14}$$

In roofline terms, the relevant machine balance becomes:

$$\mathrm{MB}_{\mathrm{disagg}} = \frac{C}{B_{\mathrm{delivered}}}.$$

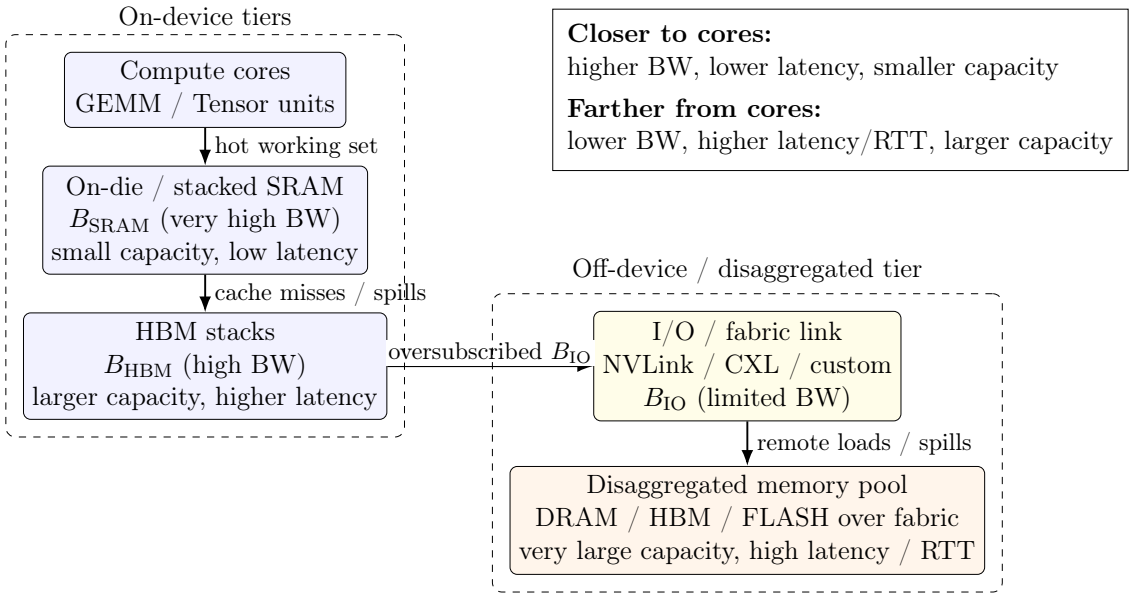

Figure 3: Tiered memory hierarchy for an accelerator: fast on-device SRAM and HBM tiers feed into an off-device, disaggregated memory pool over a bandwidth-limited I/O or fabric link.

7.1 Practical Limits: Why Disaggregated Memory Is Insufficient to Achieve Compute-Bound Transformer Training Under Current I/O Ceilings

The analysis above shows how local memory bandwidth determines whether a kernel or training step lies on the sloped (bandwidth-bound) or flat (compute-bound) portion of the roofline. We now examine whether disaggregated memory can realistically raise the effective bandwidth $B_{\text{delivered}}$ enough to move a transformer workload into the compute-bound regime.

Recall that the condition for compute-bound execution is

$$B_{\text{delivered}} \geq B_{\text{req}} = \frac{C}{\text{OI}},$$

where typical operational intensities for modern models lie in the range OI $\approx$ 30–80 FLOP/Byte. For accelerators with C = 1–10 PFLOP/s, the required bandwidth is therefore

$$B_{\text{req}} \approx 12.5\text{–}333 \text{ TB/s},$$

depending on workload structure and model architecture.

Published I/O bandwidths fall far below this threshold. Concrete, vendor-published figures for existing disaggregated-memory fabrics include:

- **CXL 2.0/3.0:** 64–128 GB/s per direction ($\ll$ 1 TB/s),
- **PCIe Gen5/6:** 64–128 GB/s per direction,
- **NVLink (Hopper/Blackwell):** 0.9–2.4 TB/s aggregate,
- **Celestial Photonic Fabric:** $\sim$ 7.2 Tbps $\approx$ 0.9 TB/s,

- **Lightmatter Passage Photonic Interconnect:** 114 Tbps ≈ 14.25 TB/s (the highest published value to date).[1]

Even under the most generous interpretation of these data, the maximum achievable delivered bandwidth into an accelerator via a disaggregated memory fabric is

$$B_{\text{delivered,max}} \approx 14 \text{ TB/s}.$$

Because this figure rests on a single vendor's published number, it is worth checking how much the conclusion depends on it. Discounting it by a factor of 2–3×, to account for the gap between a vendor specification and sustained, real-world delivered bandwidth, gives $B_{\text{delivered,max}} \approx 5$–$7$ TB/s. Recomputing the ratios below with 7 TB/s in place of 14 TB/s roughly doubles each factor (e.g., the 1.4× case becomes ≈ 2.9×, and the 9–25× case becomes ≈ 18–48×): the gap only widens under a more conservative reading of the vendor figure, so the conclusion is not sensitive to taking this number at face value.

This analysis treats the disaggregated interface as the limiting memory feed; that is, we evaluate the extreme regime in which remote memory would need to supply the full bandwidth required to reach the compute roofline.

Key Result: Even the highest published disaggregated-memory fabrics offer delivered bandwidth $B_{\text{delivered}}$ one to two orders of magnitude below the $B_{\text{req}} = C/\text{OI}$ that would be required if remote memory were the limiting tier, and therefore are insufficient to make transformer workloads compute-bound via bandwidth supplied across the disaggregated I/O interface.

$$\text{factor of } \frac{B_{\text{req}}}{B_{\text{delivered,max}}} \approx \begin{cases} 2.3\times & (\text{OI} = 30,\ C = 1 \text{ PFLOP/s}), \\ 1.4\times & (\text{OI} = 50,\ C = 1 \text{ PFLOP/s}), \\ 9\text{–}25\times & (C = 10 \text{ PFLOP/s},\ \text{OI} = 80\text{–}30). \end{cases}$$

Key Result: Even the highest published disaggregated-memory fabrics offer delivered bandwidth $B_{\text{delivered}}$ one to two orders of magnitude below the required $B_{\text{req}} = C/\text{OI}$, and therefore cannot make transformer workloads compute-bound.

7.2 A More Realistic Deployment: Disaggregated Memory as an Overflow Tier

Section 7.1 evaluates the extreme case in which disaggregated memory would need to supply the entire byte stream for a step to reach the compute roofline. In practice, no serious deployment uses disaggregated memory this way: it sits behind SRAM and HBM as a capacity overflow tier for cold or infrequently accessed bytes, such as optimizer state and checkpoint data offloaded to slower tiers [12], KV-cache pages evicted from HBM to CXL-attached memory [14], or stranded host/pooled memory reclaimed for capacity rather than bandwidth [13]. This is the same tiered role SRAM plays in front of HBM in Section 6. We can extend the blended-bandwidth model of Eq. (12) to three tiers:

$$B_{\text{eff}} \approx h_{\text{SRAM}} B_{\text{SRAM}} + h_{\text{HBM}} B_{\text{HBM}} + h_{\text{remote}} B_{\text{delivered}}, \qquad h_{\text{SRAM}} + h_{\text{HBM}} + h_{\text{remote}} = 1.$$

Returning to the GB200-class worked example of Section 6.1 ($h_{\text{SRAM}} = 0.75$), suppose a modest $h_{\text{remote}} = 0.08$ of bytes, consisting of cold or infrequently touched state rather than the hot working

[1] This figure is a vendor-published specification rather than an independently measured, deployed benchmark, and should be read with the same skepticism this note urges toward other vendor claims in Section 1. We use it anyway, as the most favorable published data point available, and check below whether the conclusion is sensitive to it.

set, are served from a disaggregated pool at the most generous published delivered bandwidth from Section 7.1, $B_{\text{delivered}} \approx 14$ TB/s, instead of from local HBM ($B_{\text{HBM}} = 8$ TB/s):

$$B_{\text{eff}} \approx 0.75(32) + 0.17(8) + 0.08(14) = 24 + 1.36 + 1.12 = 26.48 \text{ TB/s},$$

versus 26 TB/s without the disaggregated tier. Utilization rises marginally, from 29% to about 29.4%. This confirms rather than overturns the paper's central claim: at the per-step bandwidth level, disaggregated memory used as an overflow tier buys a small, real, but not transformative improvement, nowhere near enough to approach the ridge point. The gain is modest precisely because disaggregated bandwidth, even in its most generous published form, is closer to on-package HBM speed than to SRAM speed, while serving a much smaller fraction of traffic.

This does not mean the added capacity itself is unimportant; its value simply does not show up in B_{eff}. Two effects sit outside this bandwidth accounting entirely. First, additional capacity is often what allows a larger batch or longer context to be run at all, and OI itself typically increases with the batch dimension: from Eq. (8), OI_{GEMM} improves as matrices are made larger and weights are reused across more rows, so a workload that could only run at a smaller batch without extra capacity may sit at a systematically higher OI once disaggregated memory makes a larger batch feasible. This is a second-order effect the bandwidth-only model above does not capture. Second, capacity determines whether a step runs at all versus fails with an out-of-memory error, or forces additional model or pipeline parallelism purely to fit state that has nothing to do with compute throughput. Neither effect is quantified by the roofline framework used in this note, and both are exactly the kind of capacity-driven, system-level benefit flagged as out of scope in Section 1; we surface them here only to make clear that the small B_{eff} gain above is a different question from whether disaggregated capacity is useful.

7.3 Is the Gap Closing? A Generational Trend View of Fabric Bandwidth

Section 1 shows that compute has grown faster than on-package HBM bandwidth across four GPU generations (Table 1). The same question is fair to ask of off-package fabric bandwidth: is the one-to-two-orders-of-magnitude gap identified above closing over time, or widening? Table 3 tracks two representative interconnects across a comparable generational span. Per-GPU NVLink bandwidth has grown from 300 GB/s (Volta) to 1.8 TB/s (Blackwell), a 6× increase over the same four generations as Table 1, while PCIe per-lane (x16) bandwidth has roughly doubled each generation, from ∼ 16 GB/s (Gen3) to ∼ 128 GB/s (Gen6), an 8× increase spread over a longer, roughly 12–15 year span.

Interconnect	**Earliest gen.**	**Latest gen.**	**Growth**
NVLink (per GPU)	300 GB/s (Volta)	1.8 TB/s (Blackwell)	6×
PCIe (x16, one direction)	∼ 16 GB/s (Gen3)	∼ 128 GB/s (Gen6)	8×
Peak compute (Table 1)	0.125 PFLOP/s (Volta)	4.5 PFLOP/s (Blackwell)	36×

Table 3: Generational growth of representative interconnect bandwidths versus peak compute. NVLink and compute are compared over the same four GPU generations; PCIe is compared over a longer, comparable span since it advances on its own release cadence.

Both interconnects grow several times slower than compute over the same span: NVLink by roughly 6× against a 36× compute increase, and PCIe/CXL-class fabrics, which set the low end of the disaggregated-bandwidth figures in Section 7.1, by a comparable or smaller multiple over an even longer window. Extrapolating these historical growth rates, the bandwidth gap identified in

this section should be expected to persist or widen over the next several device generations, rather than close. This mirrors the conclusion Section 1 reaches for on-package HBM, now demonstrated for off-package fabric bandwidth rather than assumed.

8 Extending the Framework to Inference

Everything above is framed in terms of a "training step," but the same OI-vs.-MB logic applies to inference, with one important twist: inference is not one workload but two, with opposite intensity profiles. Modern serving systems increasingly run them on physically separate hardware pools for exactly this reason [15, 16, 17], which makes inference a useful stress test for the framework developed in this note.

8.1 Prefill: training-like, decode: nothing like it

Prefill processes an entire prompt in one parallel forward pass, batching all n prompt tokens through the same large GEMMs analyzed in Section 5. Prefill's operational intensity is governed by the same GEMM logic as Eq. (8): with enough prompt tokens to batch, weight loads are amortized across many tokens at once, and prefill sits in the same tens-to-high-OI regime as training. Nothing in Sections 6–7 needs to change to describe it.

Decode is different. Each step generates exactly one new token per sequence, autoregressively, so a batch of B sequences processes only B tokens per step, not $B \times n$ as in prefill. Model weights must still be streamed from HBM every step, but with far fewer tokens to amortize that cost over. The traffic proxy M_{tok} in Eq. (10) counts only per-token activation and KV-cache bytes; it does not include weight bytes at all, which is a reasonable simplification for a large, well-batched pass where weight traffic is negligible per token, but is exactly the term that dominates decode at low batch size. Making that term explicit: a transformer layer has on the order of $12d^2$ parameters (four $d \times d$ attention projections plus an $8d^2$ feed-forward block at $4\times$ expansion), so weight bytes per layer are approximately $W \approx 12sd^2$. For a batch of B sequences decoding at current context length n, weights are loaded once per step and shared across the batch, while activation and KV-cache traffic scale with B:

$$\text{OI}_{\text{decode}}(B, n) \approx \frac{B \cdot F_{\text{tok}}(d, n)}{12sd^2 + B \cdot M_{\text{tok}}(d, n)}. \tag{15}$$

8.2 Batch size 1 is about as memory-bound as it gets

At $B = 1$, weight bytes overwhelm the activation/KV term for any realistic d and n, so Eq. (15) reduces to $\text{OI}_{\text{decode}}(1, n) \approx F_{\text{tok}}/12sd^2 \approx \mathcal{O}(1)$ FLOP/Byte, consistent with the well-known result that single-sequence decode is close to definitionally memory-bound, since roughly the same number of bytes and FLOPs touch each weight [7]. Concretely, for $d = 4096$, $s = 2$ (BF16), and $n = 128$: $F_{\text{tok}} \approx 2.03 \times 10^8$, $12sd^2 \approx 4.03 \times 10^8$, giving $\text{OI}_{\text{decode}}(1, 128) \approx 0.5$ FLOP/Byte, one to two orders of magnitude below the 30–80 range this note otherwise treats as typical.

This also exposes a gap worth flagging in this note's own machinery: because M_{tok} omits weight bytes, $\text{OI}_{\text{tok}} = F_{\text{tok}}/M_{\text{tok}}$ from Section 5 simplifies algebraically to a constant, $2d$, independent of context length: 8192 FLOP/Byte for $d = 4096$, far above the 30–80 range Section 5 cites from empirical profiling in the same breath. The two are not actually in conflict: OI_{tok} implicitly describes the weight-amortized, large-batch limit (prefill, or decode at very large B), while the empirically observed 30–80 range reflects the batch sizes, kernel fusion overhead, and other frictions real serving systems actually operate under. But it means OI_{tok} as written should not be read as a decode-batch-

size-1 estimate; Eq. (15) is needed for that regime, and the two formulas agree only in the $B \to \infty$ limit.

8.3 How much batching does it take to look like training?

Solving Eq. (15) for the batch size at which decode reaches OI = 30 (the low end of this note's typical range) gives $B \approx 60$ at $n = 128$ and $B \approx 36$ at $n = 8{,}192$, for $d = 4096$, i.e., on the order of several tens of concurrently decoding sequences, consistent with the batch sizes production continuous-batching schedulers target for throughput. Below that, decode's required bandwidth $B_{\text{req}} = C/\text{OI}_{\text{decode}}$ is higher than anything considered in Section 7. At OI ≈ 0.5 and $C = 1$ PFLOP/s, $B_{\text{req}} \approx 2{,}000$ TB/s, roughly two orders of magnitude beyond the already-unreachable training-step figures in Section 7.1. If disaggregated bandwidth cannot feed a training step, it certainly cannot feed a low-batch decode step directly.

8.4 Where disaggregated memory actually fits: capacity for the queue, not bandwidth for the step

Decode's dependence on batch size is precisely why real inference systems do not try to feed the active decode loop from a remote memory fabric. Instead, three production and research systems converge on the same pattern this note's Section 7 predicts is the only viable one: run prefill (high-OI, training-like) and decode (low-OI, bandwidth-starved) on separately provisioned pools, and use a fast interconnect to hand off the KV cache produced by prefill to the decode pool. DistServe [15] and Splitwise [16] both disaggregate prefill and decode onto different GPUs precisely because sharing one pool forces a compromise between time-to-first-token and time-per-token that neither phase's intensity profile wants. Mooncake [17] goes further, building a KV-cache-centric store that spans GPU HBM, CPU DRAM, and SSD/RDMA-attached capacity. It uses slower, disaggregated tiers to hold KV cache for sequences that are not on the active decode critical path (queued, paused, or serving a resumed multi-turn conversation), so that the sequences actually decoding right now can stay batched large enough, and resident close enough to the compute, to sustain a workable $\text{OI}_{\text{decode}}$. This is exactly the overflow-tier role described in Section 7, now grounded in systems that are deployed in production rather than hypothesized: disaggregated memory's contribution to inference is capacity that lets the HBM-resident batch stay large, not bandwidth on the path of any single decode step.

9 Distributed Training: Communication Rooflines and Network Ceilings

Up to this point, we have focused on local memory and I/O bandwidth as the determinants of whether a training step is compute-bound. In distributed training, however, memory bandwidth is only one of two critical bandwidth channels. Even if an accelerator had sufficient SRAM/HBM or I/O bandwidth to approach the compute roofline, collective communication can impose a second and often tighter ceiling on end-to-end throughput.

9.1 Communication intensity: the network analogue of OI

The same intensity-based reasoning used for memory extends directly to communication. For a training step that exchanges S bytes of gradients or model state while executing F FLOPs, we

define the communication intensity

$$\mathrm{CI} \equiv \frac{S}{F} \qquad [\text{Byte/FLOP}], \tag{16}$$

which is the network-side counterpart to operational intensity, $\mathrm{OI} = F/M$.

A device with compute rate C and effective per-rank network bandwidth B_{net} can sustain at most

$$\mathrm{CI}_{\text{machine}} \equiv \frac{B_{\text{net}}}{C} \qquad [\text{Byte/FLOP}], \tag{17}$$

playing the same role for communication that $\mathrm{MB} = C/B$ plays for memory. Thus:

$$\mathrm{CI} \ll \mathrm{CI}_{\text{machine}} \Rightarrow \text{compute- or memory-bound}, \qquad \mathrm{CI} \gg \mathrm{CI}_{\text{machine}} \Rightarrow \text{network-bound}.$$

9.2 Collective communication and step-time ceilings

A standard approximation for an optimized all-reduce over N ranks is the (α, β) model [2]:

$$T_{\text{allreduce}}(S, N) \approx \alpha \log N + \beta S, \tag{18}$$

where α captures RTT-dominated latency and $\beta \approx 1/B_{\text{net}}$ captures the per-byte transfer cost. A training step becomes network-bound when

$$T_{\text{comm}} \gtrsim T_{\text{comp}}, \tag{19}$$

even if the device is compute-bound with respect to its local memory roofline.

As an example, consider a training step that executes $F = 10^{15}$ FLOP (one second of full-utilization compute on a $C = 1$ PFLOP/s device) while synchronizing $S = 400$ GB $= 4 \times 10^{11}$ Byte of gradient or expert-routing state per step, representative of an unsharded gradient all-reduce for a very large model, or aggregate MoE all-to-all traffic. The communication intensity is

$$\mathrm{CI} = \frac{S}{F} = \frac{4 \times 10^{11}}{10^{15}} = 4 \times 10^{-4} \text{ Byte/FLOP}.$$

With a per-rank scale-out network bandwidth of $B_{\text{net}} = 50$ GB/s $= 5 \times 10^{10}$ Byte/s (typical of a single 400 Gb/s NIC per accelerator), we obtain

$$\mathrm{CI}_{\text{machine}} = \frac{B_{\text{net}}}{C} = \frac{5 \times 10^{10}}{10^{15}} = 5 \times 10^{-5} \text{ Byte/FLOP},$$

so $\mathrm{CI} \approx 8\mathrm{CI}_{\text{machine}}$. Equivalently, $T_{\text{comp}} = F/C = 1$ s while $T_{\text{comm}} \approx S/B_{\text{net}} = 8$ s: communication takes roughly 8× longer than compute and dominates step time, even though local SRAM/HBM bandwidth is sufficient to reach the compute roof.

9.3 Interaction with disaggregated memory

It is important to note that disaggregated memory does not directly mitigate these communication limits. Even if Section 7 had shown that remote memory could supply the bandwidth needed to reach the compute roofline (it does not), distributed gradient synchronization would still impose a separate ceiling governed by CI and B_{net}. Increasing memory capacity or remote access bandwidth can change how models are sharded and what must be communicated, but without raising B_{net} it cannot, by itself, remove a regime in which network bandwidth and latency dominate step time.

9.4 Summary

Communication introduces an additional roofline, governed by CI and network bandwidth, that is conceptually parallel to the memory roofline based on OI and B. Real systems must satisfy both ceilings:

$$T_{\text{step}} = \max\{T_{\text{memory}}, T_{\text{network}}\}.$$

This dual-ceiling structure motivates the broader design implications discussed in Section 10 and highlights why raising the correct bandwidth ceiling, whether local or network, is essential for improving end-to-end training throughput.

10 Design Implications

Building on the communication roofline introduced in Section 9, we now summarize how both memory-side and network-side ceilings jointly determine end-to-end training performance. The equations above provide a short checklist for diagnosing where training is limited across compute, memory, I/O, and network:

- **Local memory vs. compute (device roofline):** Compare kernel or step OI to the machine balance $\text{MB} = C/B$ for the relevant tier (HBM, SRAM+HBM, or disaggregated memory). If $\text{OI} < \text{MB}$, the step is fundamentally bandwidth-bound at that tier; increasing C alone will not improve performance.
- **SRAM tiers:** A fast SRAM tier raises the effective bandwidth $B_{\text{eff}} \approx hB_{\text{SRAM}} + (1-h)B_{\text{HBM}}$ and thus increases utilization $U = T/C$. However, the worked example shows that even a substantial B_{SRAM} may not be enough to make realistic transformer workloads compute-bound if their OI remains well below the resulting MB.
- **Disaggregated memory and I/O ceilings:** Disaggregation primarily raises capacity. It only improves utilization in our model if the delivered bandwidth $B_{\text{delivered}} \leq \min\{B_{\text{remote mem}}, B_{\text{IO}}\}$ is large enough that $\text{MB}_{\text{disagg}} = C/B_{\text{delivered}}$ approaches the OI of real workloads, and even then the gain is bounded by the fraction of traffic that can be served without making the disaggregated interface the active limiting tier. When $\text{OI} \ll \text{MB}_{\text{disagg}}$, the system remains memory-bound regardless of how large the remote pool is.
- **Network collectives (communication roofline):** For distributed training, the (α, β) all-reduce model $T_{\text{allreduce}}(S, N) \approx \alpha \log N + \beta S$ and the communication intensity

  $$\text{CI} = \frac{\text{Bytes communicated}}{\text{FLOPs executed}}$$

  play the same role on the network that OI does for memory. Comparing CI to the machine-level quantity

  $$\text{CI}_{\text{machine}} = \frac{B_{\text{net}}}{C}$$

  tells us when network bandwidth and latency become the dominant bottlenecks. As shown in Section 9, high CI can make a step communication-bound even if local memory bandwidth is sufficient to approach the compute roofline. This mirrors the structure of the memory roofline: low OI places a step on the memory-bound slope, while high CI places it on the communication-bound slope.

From a system-design perspective, every proposed feature, including additional HBM stacks, an SRAM tier, disaggregated memory, in-memory primitives, in-network aggregation, or more network bandwidth, should be evaluated in terms of:

> Which ceiling does this raise (compute, local memory bandwidth, off-package I/O, or network bandwidth), and is that ceiling the dominant limiter for the workload's operational and communication intensities?

Memory-side ceilings (OI vs. MB) and network-side ceilings (CI vs. $\mathrm{CI}_{\mathrm{machine}}$) are largely independent at the hardware-ceiling level. Improving one generally does not remove the other: as Section 9 demonstrated, even idealized memory capacity and bandwidth cannot overcome a communication-bound regime. Effective system design therefore requires identifying which ceiling is currently active for a given workload and parallelization strategy, and raising the one that actually limits step time.

11 Future Work and Open Questions

This note has deliberately focused on intensity-based upper bounds using simplified bandwidth models for local and disaggregated memory, and has omitted several important practical effects (latency and RTT sensitivity, bandwidth-delay product constraints, queuing and prefetch depth, granularity and coherence behavior, and memory reliability and error rates) in order to maintain analytical clarity.[2] These omissions affect the two ceilings differently. For the pure bandwidth ceilings (B_{HBM}, B_{SRAM}, B_{IO}, B_{net}), they are genuinely conservative in the sense we intend: no amount of scheduling or software cleverness lets a system move more bytes per second than a link's physical bandwidth allows, so ignoring queuing or error-rate overhead only removes headroom, never adds it. For the latency-driven terms specifically (α, RTT, bandwidth-delay product), the direction is less clean: real systems use prefetching, double buffering, and compute/communication overlap precisely to hide these costs behind useful work, so a naive model that ignores them is not guaranteed to be conservative in the same sense; it may simply be omitting a term that well-engineered software already hides. We flag this distinction explicitly because it is easy to conflate "bandwidth-conservative" with "conservative in every respect," and only the former is supported by the argument in this note. These choices leave a number of questions open for future work:

- **Trace-driven tiered-memory analysis.** The present model treats each memory or I/O tier as a single limiting roofline. An important next step is to incorporate measured traces from large-model training and inference runs to quantify the fraction of bytes served by each tier (SRAM, HBM, and disaggregated memory), and to evaluate step time using a simple multi-term bound such as

$$T_{\mathrm{step}} \geq \max\left(\frac{F}{C}, \frac{M_{\mathrm{SRAM/HBM}}}{B_{\mathrm{SRAM/HBM}}}, \frac{M_{\mathrm{remote}}}{B_{\mathrm{IO}}}\right).$$

  This would more directly characterize regimes where disaggregated memory is used as an overflow tier rather than as the primary bandwidth source.

- **Latency, BDP, and outstanding concurrency.** While we model B_{IO} as an idealized peak, real fabrics are limited by RTT, bandwidth-delay product, and finite queue depth. A useful extension would couple the intensity view with a simple model of outstanding requests

[2] See the discussion of scope and simplifying assumptions in Section 1.

and prefetch depth, quantifying the delivered bandwidth as a function of RTT and access pattern, and identifying the break-even points where remote memory transitions from tolerable to dominant in step time.

- **Kernel mix and intensity distributions.** We treat transformer training steps using a single effective operational intensity in the "tens of FLOP/Byte" range. Future work should report the distribution of OI across the constituent kernels and phases (e.g., matmuls versus softmax, layernorm, and elementwise operations), and apply the roofline analysis at this finer granularity to understand which phases are most sensitive to bandwidth at each tier.

- **Coupled memory-communication behavior in distributed training.** The current communication roofline treats communication intensity CI as a property of the workload independent of local memory capacity. In practice, changes in per-rank memory capacity (e.g., via disaggregation) can alter sharding strategies, checkpointing policies, and gradient accumulation, thereby changing the communicated volume S and effective CI. Trace-driven studies that jointly vary memory capacity, parallelization strategy, and network bandwidth would clarify how often capacity-oriented features indirectly shift the active communication ceiling.

- **Cost- and energy-normalized ceiling comparisons.** Finally, the framework here is expressed in physical units (FLOP/s, Byte/s) rather than in $ or W. A natural extension is to integrate cost and power models for additional HBM stacks, SRAM tiers, disaggregated memory fabrics, and network upgrades, and to compare these options on performance-per-dollar and performance-per-watt for representative ranges of OI and CI.

Overall, these directions would retain the simplicity of the intensity-based view while bringing it closer to end-to-end system behavior, especially in regimes where disaggregated memory is used primarily as a capacity extension rather than a replacement for on-package bandwidth.

12 Conclusion

This note has developed a unified, intensity-based framework for reasoning about the performance limits of large-scale AI training across compute, memory, I/O, and network domains. By expressing kernels and training steps in terms of their operational intensity (OI) and communication intensity (CI), we can evaluate any architectural change with respect to the ceilings it must overcome: compute throughput C, local memory bandwidth B, off-package I/O bandwidth B_{IO}, and network bandwidth B_{net}.

For device-local execution, the roofline model shows that workloads with OI $<$ MB $= C/B$ are fundamentally memory-bound. Even with large SRAM tiers or multiple HBM stacks, realistic transformer OI values remain far below the machine balance of modern accelerators. As shown in Section 7, disaggregated memory increases capacity but does not deliver sufficient bandwidth to reach the compute roofline, leaving such workloads in a memory-bound regime.

Section 9 introduced the communication roofline, the direct analogue of the memory roofline for distributed training. When the communication intensity CI exceeds the machine-level ratio B_{net}/C, or when RTT inflates the latency term α, collective communication becomes the dominant limiter even if local memory bandwidth is ample. In multi-site deployments, these effects are further amplified by heterogeneous RTTs and WAN bandwidth ceilings.

Taken together, these results highlight a central theme: meaningful performance gains require raising the correct ceiling for the workload's intensities. Additional FLOPs, larger memory pools, higher-capacity disaggregated memory systems, or more complex interconnects provide benefit only

to the extent that they lift the active ceiling in either the memory or communication roofline. The framework developed here provides a quantitative basis for evaluating architectural decisions, memory hierarchies, network designs, and multi-site training strategies in a consistent and comparable manner. The bandwidth ceilings themselves are conservative upper bounds: no scheduling or software technique can exceed a link's physical throughput. The latency-driven effects we neglect, including RTT, BDP, and queue depth, are not guaranteed to move the picture in the same direction, since real systems can partially hide them through overlap and prefetching; we omit them for analytical clarity, not because ignoring them is safely one-sided.

References

[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures," *Communications of the ACM* (preprint / technical report), 2009.

[2] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of Collective Communication Operations in MPICH," *International Journal of High Performance Computing Applications*, 2005.

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019.

[4] H. de Vries, "In the long (context) run," blog post, 2024. Available online.

[5] JAX/Google, "All the Transformer Math You Need to Know," *How To Scale Your Model* notes, 2025. Available online.

[6] F. Timbers, "Where do LLMs spend their FLOPS?," blog post, 2024. Available online.

[7] Kipply, "Transformer Inference Arithmetic," blog post, 2022. Available online.

[8] Baseten, "A guide to LLM inference and performance," blog post, 2025. Available online at baseten.co.

[9] ObjectiveMind.AI, "Memory Bandwidth Engineering: The True Bottleneck in LLM GPU Architecture," blog post, 2025. Available online at objectivemind.ai.

[10] S. Kim et al., "Full Stack Optimization of Transformer Inference: A Survey," arXiv:2302.14017, 2023.

[11] A. Gholami et al., "AI and Memory Wall," *IEEE Micro*, 2024.

[12] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning," *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021.

[13] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, and R. Bianchini, "Pond: CXL-Based Memory Pooling Systems for Cloud Platforms," *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, 2023.

[14] Y. Tang et al., "Exploring CXL-based KV Cache Storage for LLM Serving," *ML for Systems Workshop, NeurIPS*, 2024.

[15] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, 2024.

[16] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," *51st Annual International Symposium on Computer Architecture (ISCA '24)*, 2024.

[17] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving," *23rd USENIX Conference on File and Storage Technologies (FAST '25)*, 2025.